

CSS311



Parallel and Distributed Computing

B.Tech CSE | Batch 3 | Semester V | Odd Semester 2026-27

Dr. Mirothali Chand
Dept. of Computer Science & Engineering -2
IIIT Kottayam.

Syllabus

Credits: 4 [L:3 T:0 P:2]

- **Architectures** – Multi-core and Many-core architectures, Accelerators (SIMD units -Vectorization, GPUs), Goals of parallel systems.
- **Applications** – Scientific applications, Characteristics, requirements, regular grid applications, irregular applications, data dependence, parallelization process.
- **Parallel Programming on Shared Memory** – OpenMP (C++ languages), Execution Model, Shared and private data, Directives, Barriers, Sections, Run-Time library functions, scheduling strategies, Scalability study, OpenMP for accelerator programming.
- **Parallel Programming on Distributed Memory** – MPI, Collective operations, Non-Blocking Collectives, Process topologies, Parallel I/O, Single sided communications.
- **Performance Tools** – Concepts, Event-Model execution, profiling, tracing, types of profiling, profiling tools – Scalasca, Score-P, MPI-P, Energy Analyzer, Tracing tools, Autotuning – Periscope Tuning Framework.

Course Structure

5 Units covering the full parallel computing stack:

Unit 1: Architectures (WHY parallelism, HOW hardware supports it)

Unit 2: Applications (WHAT kinds of problems benefit)

Unit 3: OpenMP (HOW to program shared memory systems)

Unit 4: MPI (HOW to program distributed memory systems)

Unit 5: Performance Tools (HOW to measure and improve)

By the End of This Course, You Will Be Able To

- **Write** a [parallel C program](#) that uses all cores on your laptop → Unit 3
- Write a [distributed C program](#) that runs across multiple machines → Unit 4
- [Predict the maximum speedup](#) of a program before writing it → Unit 1
- Identify [why a parallel program is slower](#) than expected → Unit 5
- [Design a parallelization strategy](#) for any given problem → Unit 2
- [Read and interpret performance](#) profiling reports → Unit 5
- Understand [how GPU computing works](#) and when to use it → Unit 1

Assessment Structure

Assessment Category	Component		Weight Within Category	Overall Weight
Theory Evaluation (70%)	Mid-Semester Exam		25%	17.5%
	Continuous Evaluation	2 Quizzes	30%	21.0%
		2 Assignments		
	End-Semester Exam		45%	31.5%
Lab Evaluation (30%)	Regular lab (algo, code & report)		50%	30.0%
	Final lab (viva & lab exam)		50%	
Total	—		—	100%

Attendance requirement = 80%

Sequential and Parallel

What each means, how they differ, and where we see them every day

Sequential — One Thing at a Time, in Order

Making a cup of tea: you do each step only after the previous one is done.

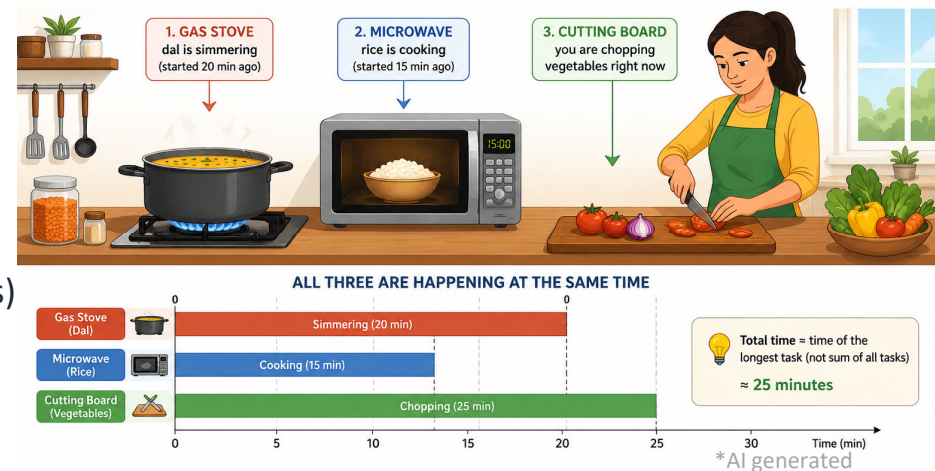
- Step 1: **Boil water** → wait for it to finish
- Step 2: Add **tea leaves to the cup** → only after water is ready
- Step 3: Pour boiling water into the cup
- Step 4: Wait 3 minutes for the tea to brew
- Step 5: Add milk and sugar
- → Total time = sum of all steps → cannot skip ahead
- Key point: each step depends on the previous step being complete

ANALOGY

Parallel — Multiple Things Happening at the Same Time

Cooking a full meal: you use the stove, microwave, and cutting board simultaneously.

- **Gas stove:** dal is simmering (started 20 min ago)
- **Microwave:** rice is cooking (started 15 min ago)
- **Cutting board:** you are chopping vegetables right now
 - All three are happening at the same time
- **Total time** $\approx$ time of the longest task (not sum of all tasks)



- **Key point:** these tasks are **independent** — dal does not wait for rice
- If **one task NEEDS** the result of another, it must wait — that is the limit

Sequential Computing — The Formal Definition

Sequential computing: a single processor executes instructions one after another, in a fixed order, where each instruction completes before the next begins.

- One processor does all the work
- Instructions are executed in order: instruction **1** → **2** → **3** → ...
- No instruction starts until the previous one finishes
- This is how every C program you have written so far works

```
for(i=0; i<8; i++) C[i] = A[i] + B[i];
```

Step i=0 runs → completes → Step i=1 runs → completes → ...

- **8 additions = 8 separate steps = at least 8 clock cycles**

Sequential Code — What Really Happens

```
int A[8] = {10, 20, 30, 40, 50, 60, 70, 80};
int B[8] = { 1,  2,  3,  4,  5,  6,  7,  8};
int C[8];

for (int i = 0; i < 8; i++)
    C[i] = A[i] + B[i];    /* one addition at a time */
```

Execution trace — each step runs and completes before the next:

i	Operation	When
i = 0	$C[0] = 10 + 1 = 11$	Clock cycle 1-4
i = 1	$C[1] = 20 + 2 = 22$	Clock cycle 5-8
i = 2	$C[2] = 30 + 3 = 33$	Clock cycle 9-12
...	...and so on for i=3 to 7	8 steps total

Parallel Computing — The Formal Definition

Parallel computing: using two or more processors simultaneously to solve a single problem faster, by dividing the work into parts that are solved at the same time.

- **More than one processor** works on the problem at the same time
- The **problem is divided into** parts — each part goes to one processor
- All parts are **computed simultaneously** (or overlapping in time)
- **Results from all parts are combined** to produce the final answer
- The same 8-element addition with 4 processors:

Processor 0: computes $C[0]$, $C[1]$ (at the same time as Processor 1)

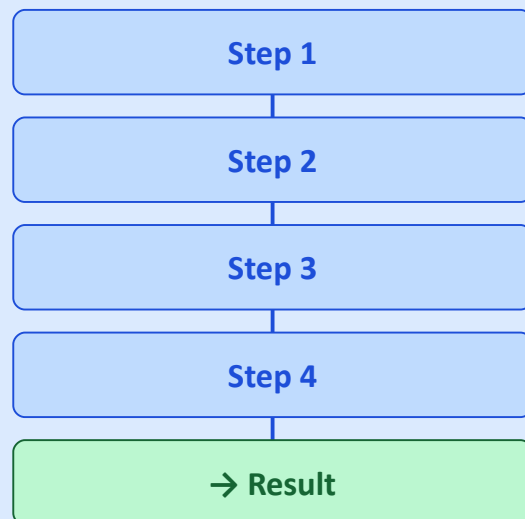
Processor 1: computes $C[2]$, $C[3]$

Processor 2: computes $C[4]$, $C[5]$

Processor 3: computes $C[6]$, $C[7]$

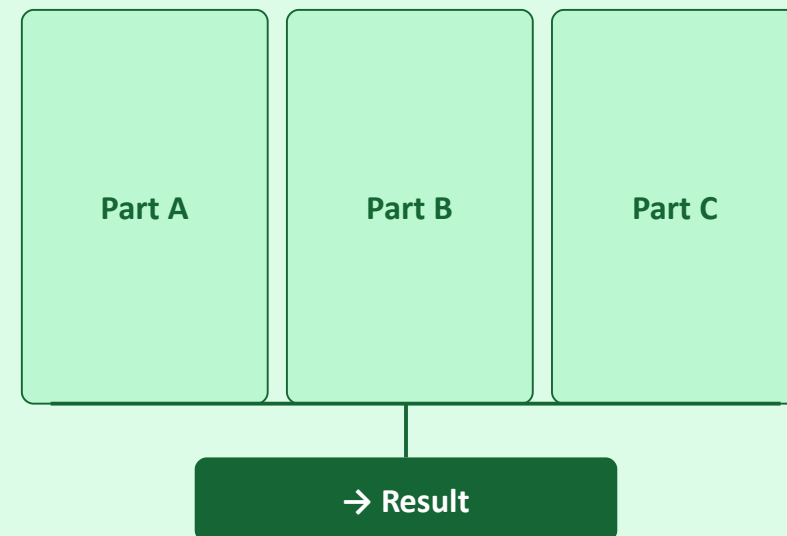
Sequential vs Parallel — Side by Side

SEQUENTIAL



One step at a time

PARALLEL



Work divided, done simultaneously

What is a Distributed System?

A distributed system is a collection of independent computers that appear to the user as a single coherent system — even though they are separate machines connected by a network.

- The key word: **SEPARATE machines** — each has its own processor and its own memory
- They **communicate ONLY by sending messages** over a network
(Unlike a multi-core CPU where all cores share the same RAM)
- From the outside, it behaves like one system — **one Google, one WhatsApp**
- Three essential characteristics:
 1. **Multiple autonomous computers** — each can fail independently
 2. **Network connects them** — communication takes time (milliseconds, not nanoseconds)
 3. **No shared memory** — one machine cannot directly read another machine's RAM

Parallel vs Distributed — The Core Difference

PARALLEL COMPUTING

- One machine, many cores
 - All cores share the SAME RAM
 - Communication = read/write shared variable
 - Speed: nanoseconds (< 100 ns)
 - Scale: 2 to ~128 cores typically
 - Risk: race conditions
- Example: [your laptop doing video render](#)
- Analogy: 4 chefs in ONE kitchen

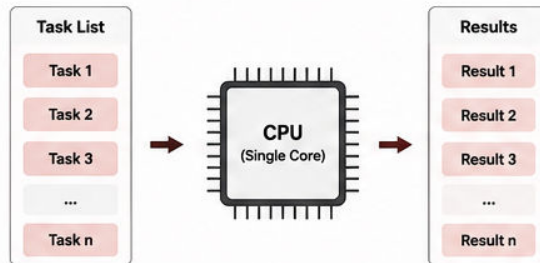
DISTRIBUTED COMPUTING

- Many machines, connected by network
 - Each machine has its OWN private RAM
 - Communication = send message over network
 - Speed: milliseconds (1,000,000 ns)
 - Scale: 2 to millions of machines
 - Risk: deadlocks and partial failures
- Example: [Google searching across 1000 servers](#)
- Analogy: 4 chefs in 4 SEPARATE buildings

Sequential vs Parallel vs Distributed

1) SEQUENTIAL COMPUTING

One processor executes one task at a time



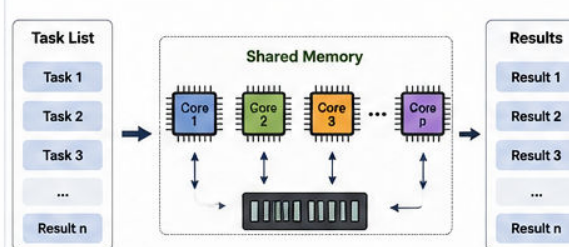
Tasks are executed one after another.
Total time = sum of all task times

Timeline



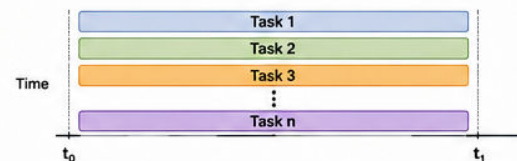
2) PARALLEL COMPUTING

Multiple processors (cores) work on tasks simultaneously on the same machine



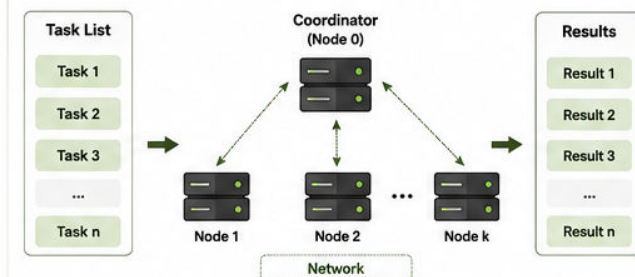
Tasks are executed at the same time using multiple cores.
Total time $\approx$ time of the longest task

Timeline



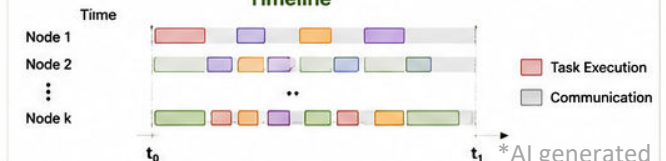
3) DISTRIBUTED COMPUTING

Multiple computers (nodes) work on tasks simultaneously over a network



Tasks are executed on different machines. Results are sent back to the coordinator.
Total time $\approx$ time of the longest task + communication overhead

Timeline



* AI generated

2

Where Do You See Parallel Computing Every Day?

You are already using it — you just did not know the name for it

Real World Example 1 — WhatsApp Video Call

When you are on a video call right now, your phone does ALL of this at the same time:

- **Camera capture:** reads 30 frames per second from the camera sensor
- **Video compression:** compresses each frame using H.264 codec
- **Encryption:** encrypts compressed data for security
- **Network send:** transmits encrypted packets over Wi-Fi or 4G
- **Network receive:** receives incoming video packets from the other person
- **Decryption + decompression:** unpacks and decompresses the incoming video
- **Display rendering:** shows the decoded video on your screen

→ All 7 of these run simultaneously → This is parallel computing

Real World Example 2 — Google Search and Instagram Filters



Two different types of parallelism — one is task-based, one is data-based:

Google Search: you type one query

- → That query is sent to thousands of servers simultaneously
- → Each server checks a different portion of the web index
- → All results come back and are ranked in 0.2 seconds
- → Without parallelism: checking 8 billion pages one at a time would take years

Instagram Filter: a blur filter is applied to a 12-megapixel photo

- → 12 million pixels — same computation (blur) applied to every pixel
- → GPU processes thousands of pixels simultaneously
- → Without parallelism: applying the filter pixel by pixel would take minutes

Real World Example 3 — ChatGPT and PARAM Siddhi

The largest-scale parallel computing in the world — running right now:

PARAM Siddhi — India's AI Supercomputer (C-DAC Pune):

- Performance: 4.6 PetaFLOPS (4.6×10^{15} calculations per second)
- Used for: monsoon prediction, drug discovery, material science
- The flood warning system that protects Kerala uses models like this



Performance:
4.6 PetaFLOPS
(4.6×10^{15} calculations per second)



Used for: **monsoon prediction, drug discovery, material science**



The **flood warning system** that protects **Kerala** uses models like this



PARAM Siddhi
India's Most Powerful AI Supercomputer

Real-World Applications



Monsoon Prediction



Drug Discovery



Material Science

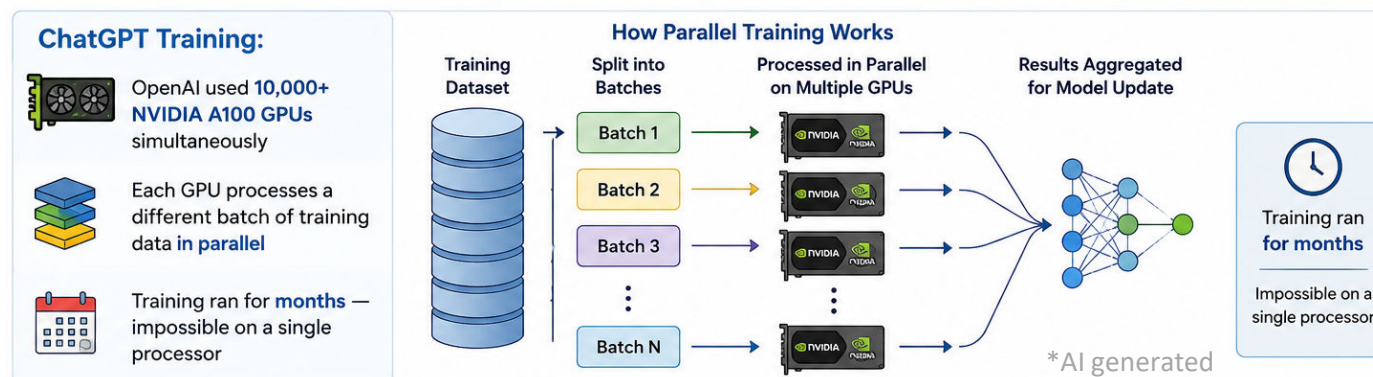
*AI generated

Real World Example 3 — ChatGPT and PARAM Siddhi

The largest-scale parallel computing in the world — running right now:

ChatGPT Training:

- OpenAI used 10,000+ NVIDIA A100 GPUs simultaneously
 - Each GPU processes a different batch of training data in parallel
 - Training ran for months — impossible on a single processor
- **Every application that matters today runs on parallel computing**



The Pattern Across All These Examples

Every example has the same structure:

- ONE problem → divided into MANY parts → solved SIMULTANEOUSLY → combined
- The benefit: the problem is solved faster than one processor could alone
- The condition: the **parts must be independent (or mostly independent)**
- If Part B needs the result of Part A, then B must wait for A
- That waiting reduces the parallelism benefit
- This **condition — independence — is the central challenge of parallel programming**

Distributed Systems You Use Every Day

- **Google Search** — you type a query, it goes to thousands of servers simultaneously
 - Each server checks a different slice of the web index
 - Results come back from all servers and are ranked — in 200 milliseconds
 - Your query crossed a network, went to multiple machines, came back
- **WhatsApp** — when you send a message, it travels across multiple servers
 - Authentication server: verifies your identity
 - Message server: stores and routes the message
 - Notification server: wakes up the recipient's phone
 - All three are separate machines communicating over a network
- **UPI / BHIM / Google Pay** — one payment involves multiple banks' servers
 - Your bank → NPCI clearing server → recipient's bank → confirmation back
 - Four different organisations, four different machines, one transaction
- **Netflix India** — when you press play, the video comes from a CDN server near you
 - Netflix has servers in Mumbai, Chennai, Bengaluru — the nearest serves your request

Class 2

Parallel Computing. Why?

From the Von Neumann machine to modern multi-core processors — the full story

1

Before Parallel — The Computer That Started It All

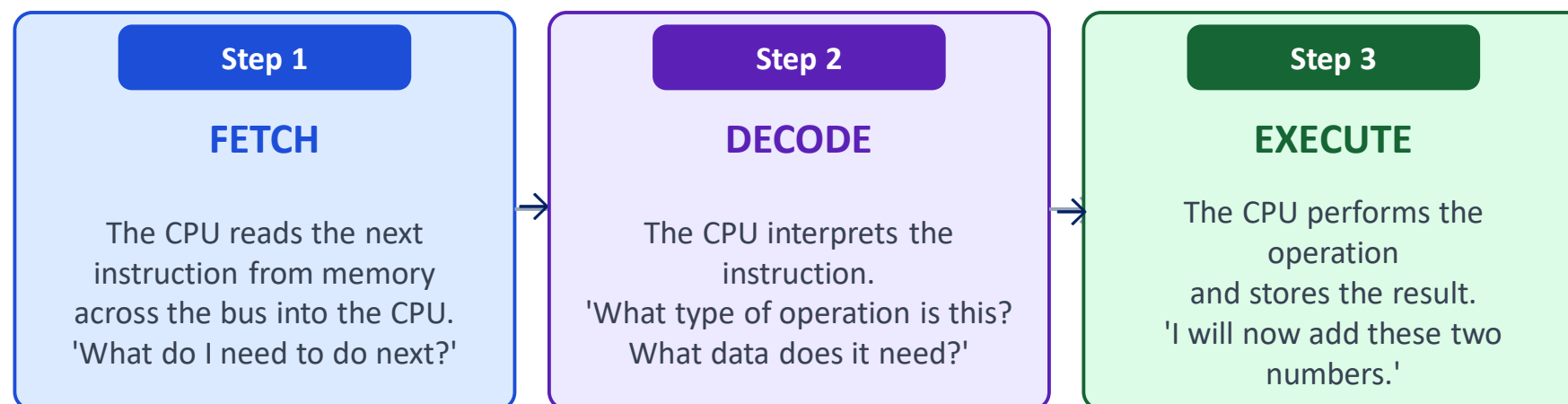
We need to understand how computers worked before parallelism existed

The Von Neumann Architecture — Where It All Began



- In 1945, mathematician **John von Neumann** described the design of a modern computer
- His design is called the **Von Neumann Architecture**
- Almost every computer built **from 1945 to today** still follows this basic design
- It has THREE main components:
 1. **The Processor (CPU)** — does all the computation
 2. The **Memory** — stores instructions and data
 3. The **Bus** — the pathway connecting CPU and memory
- These three components interact in a simple cycle, **repeated billions of times per second**

The Fetch-Decode-Execute Cycle — What the CPU Does



This cycle repeats for every single instruction in your program

- A simple for-loop with 100 iterations runs through this cycle hundreds of times

Key bottleneck: the FETCH step requires data to travel from Memory to CPU across the Bus

- If memory is slow, the CPU waits — this becomes critical as processors get faster
- This waiting is one of the fundamental reasons for the **Memory Wall**

From 1971 to 2004 — The Era of Free Performance

- Gordon Moore (Intel **co-founder**, 1965): transistor count on a chip doubles every ~2 years
- This observation, called **Moore's Law**, held true for 40 years
- More transistors → higher clock speed → faster execution

1971: Intel 4004 → 0.74 MHz (740,000 cycles per second)

1985: Intel 386 → 16 MHz

1993: Intel Pentium → 60 MHz

2000: Intel Pentium 4 → 1.5 GHz (1,500,000,000 cycles per second)

2004: Intel Pentium 4 Prescott → 3.8 GHz ← this is where it **STOPPED**

2

2004 — The Year the Free Lunch Ended

Three walls stopped single-core speed from growing further

Double the clock speed → roughly 8× the power consumption. Physics stops you.

- **Transistors** switch on and off billions of times per second to perform computation
- Each switch consumes a tiny amount of electrical power and generates heat
- More switches per second (higher clock speed) = more power = more heat
- The **relationship**: $\text{Power} \propto \text{Capacitance} \times \text{Voltage}^2 \times \text{Frequency}$

Doubling frequency roughly requires raising voltage too → power cubes

- Intel Pentium 4 at 3.8 GHz already consumed 115 Watts — very hot

A 10 GHz version of the same design would need water-cooling the size of a household refrigerator — not practical

- **Heat dissipation**, not transistor count, became the hard physical limit

$$\text{Power} \propto C \times V^2 \times f \quad (C = \text{capacitance}, V = \text{voltage}, f = \text{frequency})$$

CPU speed grew 60%/year. Memory bandwidth grew only 10%/year. A 330× gap opened.

- From **1980 to 2000**, CPU performance improved at ~60% per year
- Memory bandwidth (how fast data travels from RAM to CPU) improved at only ~10% per year
- After 20 years: this **gap became enormous**
- Today: a modern CPU core completes an operation in $\approx$ **0.3 nanoseconds (ns)**
 - But fetching data from RAM takes $\approx$ **100 nanoseconds**
 - That is a 330× difference in speed
- Without anything to bridge this gap:
 - The processor would be idle 99% of the time, waiting for data
- **Caches were invented to bridge this gap**

CPU operation: 0.3 ns → RAM access: 100 ns → 330× slower

Processors already automatically exploit instruction-level parallelism — and have hit its limits.

- ILP = **Instruction-Level Parallelism**
- Modern processors already try to run multiple instructions simultaneously
 - They use techniques like like **pipelining** (overlapping stages of different instructions), **out-of-order execution** (executing instructions in a different order than written, when safe), and **branch prediction** (guessing which way an if-else will go, to avoid stalling).
 - These techniques find independent instructions and run them at the same time
 - All of this **happens automatically** — you do not write any special code
- The problem: in typical sequential code, only 3-5 instructions are **independent at any given moment** — the rest have dependencies
- Adding more hardware to find ILP gives rapidly diminishing returns

The cost grows faster than the benefit — the wall is real

```
Pipelining: while instruction 1 is in the Execute stage, instruction 2 is  
being Decoded
```


Three Walls — Why Single-Core Speed Cannot Grow Further

1

Power Wall

More speed needs exponentially more power. Heat becomes unmanageable. Physics limits this.

2

Memory Wall

CPU is 330× faster than RAM can supply data. Processor spends most time waiting.

3

ILP Wall

Processors already extract all available automatic parallelism from sequential code. Ceiling reached.

All three walls hit simultaneously around 2004 — single-core clock speed has barely changed since.

The Solution — More Cores Instead of Faster Cores

- Intel's answer in **2006**: the Core 2 Duo processor

Previous best: Intel Pentium 4 Prescott — 1 core at 3.8 GHz

Core 2 Duo: 2 cores at 2.66 GHz ← clock speed DROPPED

- Why did clock speed drop? To stay within **safe power and heat limits**
- But now there are TWO workers instead of ONE

Ideal case: 2 independent tasks → each core handles one → done in half the time

- The **important condition**: this only helps if your software can USE both cores
 - A sequential program on Core 2 Duo: uses 1 core at 2.66 GHz, not 3.8 GHz
 - A parallel program on Core 2 Duo: can use both cores simultaneously

The Shift — 20 Years of Core Count Growth

Year	Processor	Cores	Clock Speed	What this shows
2004	Intel Pentium 4 Prescott	1	3.8 GHz	Peak single-core era
2006	Intel Core 2 Duo	2	2.66 GHz	Clock speed DROPPED, cores doubled
2011	Intel Core i7-2600K	4	3.4 GHz	Quad-core becomes mainstream
2017	AMD Ryzen 7 1800X	8	3.6 GHz	8-core consumer CPU
2019	AMD Ryzen 9 3950X	16	3.5 GHz	16-core consumer desktop
2022	AMD EPYC 9654	96	2.4 GHz	96-core server processor
2024	Intel Core i9-14900K	24	3.2 GHz	Hybrid design, 24 cores

From Desktop to Supercomputer



Personal Devices

- Laptop: 4-16 cores
- Smartphone: 4-8 cores
- Gaming PC: CPU (8-24 cores) + GPU (thousands of cores)
- Even a Raspberry Pi has 4 cores
- **Key:** Parallelism is everywhere, not just supercomputers

Supercomputers

- **Frontier** (USA): 8.7M cores, 1.2 ExaFLOPS
- **PARAM Siddhi** (India): 4.6 PetaFLOPS
Used for: climate, defence, genomics
- **India's NSM** (National Supercomputing Mission) building 73 systems
- **Key:** Same parallel principles at every scale

DO THIS



How many cores does your laptop have?

Check it :

- Windows: Task Manager → Performance tab → CPU → Cores
- Mac: Apple menu → About This Mac → System Report → Hardware Overview
- Linux: Open terminal and type: `nproc`
- Colab: Run: `!nproc` in a code cell

Parallel Computing — The Scale in 2024

8–16

Cores in a typical laptop
CPU today

16,384

CUDA cores in NVIDIA RTX
4090 GPU

10,000+

GPUs used to train ChatGPT

8.7 M

Cores in Frontier — world's
fastest supercomputer

4.6 PF

PARAM Siddhi performance
(C-DAC Pune)

1

Core used by a sequential C
program

Why This Matters to You as a Programmer

- Sequential code no longer gets faster just by buying a new computer

Your sorting algorithm runs at the same speed in 2024 as 2014

Because: same clock speed, same sequential execution, 1 core used

- Parallel code benefits directly from every new core

A parallel sort on 8 cores can run up to 8× faster than sequential

- All modern software that matters is parallel:

Web browsers, databases, OS schedulers, AI inference, game engines

**Your laptop has 8 cores.
If we could make all 8 work together,
would your program run exactly 8× faster?**

- Think about: what if some parts of the program cannot be split?
- Think about: what if all 8 cores need to read from the same memory?
- Think about: what if 7 cores finish early and must wait for the 8th?

KEY TAKEAWAYS

1. The **Von Neumann computer**: one processor + one memory + one bus — this is the foundation
2. **Moore's Law**: transistors doubled every 2 years — clock speed followed until 2004
3. **Three walls** ended the single-core era: Power (heat limit), Memory (330× gap), ILP (ceiling)
4. **Industry response in 2006**: more cores instead of faster cores (Core 2 Duo)
5. **Consequence**: sequential code does not benefit from additional cores
6. **Parallel code can use all available cores** — this course teaches you how to write it

NEXT: what is inside each core? How does the cache hierarchy work? Why does the access pattern in your code matter? And what is false sharing — the hidden parallel performance trap?

Class 3

FROM PREVIOUS SESSIONS

1. The **Von Neumann computer**: one processor + one memory + one bus — this is the foundation
2. **Moore's Law**: transistors doubled every 2 years — clock speed followed until 2004
3. **Three walls** ended the single-core era: Power (heat limit), Memory (330× gap), ILP (ceiling)
4. **Industry response in 2006**: more cores instead of faster cores (Core 2 Duo)
5. **Consequence**: sequential code does not benefit from additional cores
6. **Parallel code can use all available cores** — this course teaches you how to write it

Multi-core CPU Architecture

What is inside your processor — and why it shapes every parallel program you write

Where We Are — Building on What We Know

WHY parallel computing became necessary?

- Three walls stopped single-core speed from growing after 2004
- The industry responded by adding more cores instead of making faster cores

Now the question is: what exactly is a core?

- What components are inside each core?
- How do multiple cores on the same chip share resources?
- Why does the way you write code affect how fast it runs on this hardware?

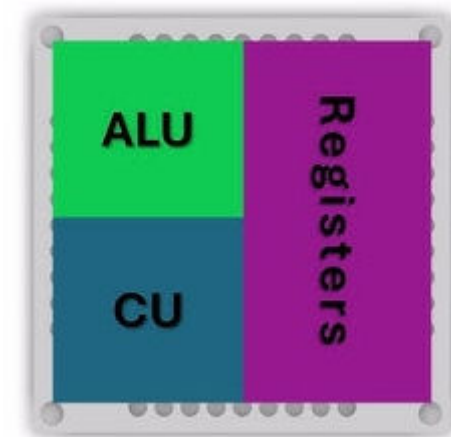
1

Inside One CPU Core

Every core is a self-contained processing unit — let us open it up

A CPU Core — The Three Essential Parts

- A CPU core is the **fundamental unit** that executes your program's instructions
- Every core contains three essential components:
 1. **Control Unit (CU)** — the manager
 - Reads instructions from memory, decides what to do next
 - Does NOT do computation — only directs traffic
 2. **Arithmetic Logic Unit (ALU)** — the worker
 - Performs **actual computations**: addition, subtraction, comparison
 - This is where $C[i] = A[i] + B[i]$ actually happens
 3. **Registers** — the notepad
 - **Tiny storage locations** built directly on the processor chip
 - Hold the data the ALU is currently working on



Registers — The Fastest Storage in the Computer

A register is a tiny storage location built directly inside the processor chip, capable of being accessed in less than 1 clock cycle (within nanos secs) — essentially instant.

- A typical **CPU core** has **16 to 32** general-purpose registers
- Each **register holds one value** — typically **64 bits (8 bytes)** on a modern CPU
- **Access time: < 1 nanosecond** — faster than any other form of storage
- **Why so fast?** They are **wired directly to the ALU** — no bus, no wait
- **Why so few?** They are **expensive** to build on silicon
- When you write: `int x = A[i] + B[i];`

The compiler loads A[i] into Register R1

Loads B[i] into Register R2

Instructs ALU: add R1 + R2, store result in R3

R3 is then written back to memory as x

The ALU — Where Computation Happens

The Arithmetic Logic Unit (ALU) is the component that performs all computations — additions, subtractions, multiplications, and logical comparisons.

- The ALU takes two inputs from registers and produces one **output**
- Operations it can perform:
 - **Arithmetic:** addition, subtraction, multiplication, division
 - **Logical:** AND, OR, NOT, XOR (used in conditions like if statements)
 - **Comparison:** equal to, greater than, less than (used in loop conditions)
 - **Shift:** multiply/divide by powers of 2
- Example: the line $C[i] = (A[i] + B[i]) / 2$ requires the ALU to:
 - **Step 1:** add $A[i]$ and $B[i]$ → store in a register
 - **Step 2:** shift right by 1 (divide by 2) → store result
- The ALU does ONE operation per clock cycle (in a simple model)

2

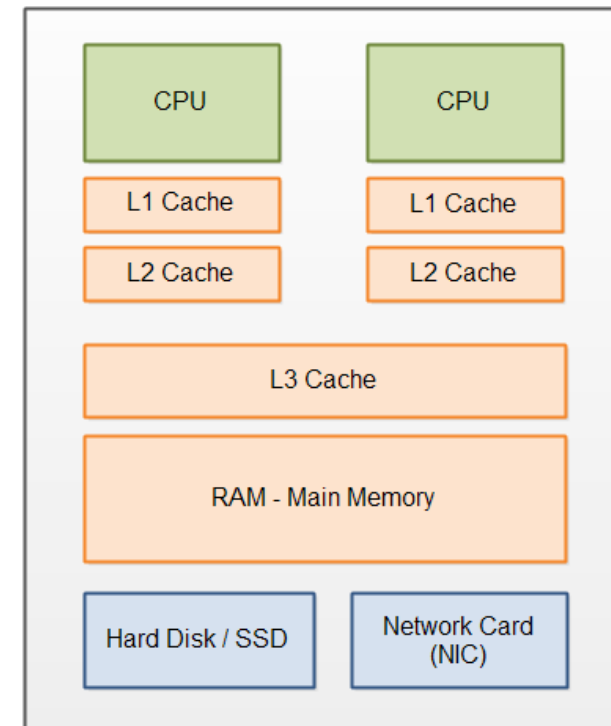
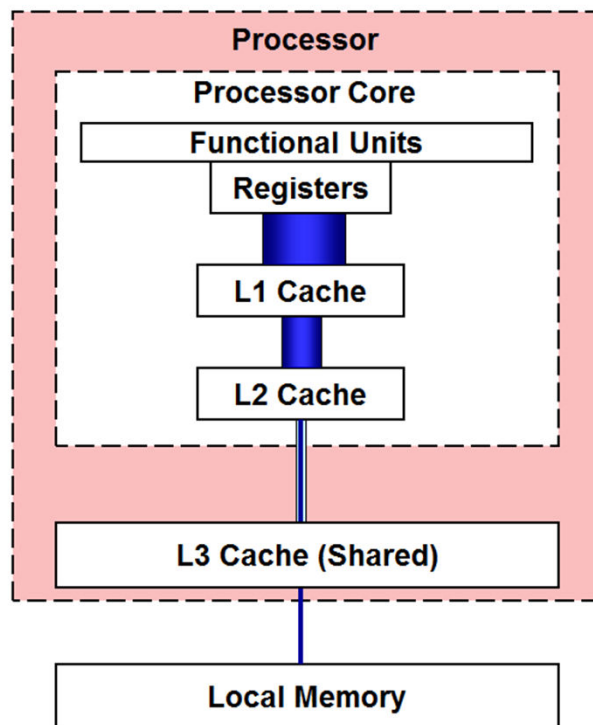
Multi-core — Duplicate the Core

Same complete unit, many times, on one chip

Multi-core Processor — The Same Core, Repeated

- A multi-core processor contains **multiple complete CPU cores** on a single chip
- Each core has its **own: ALU, Control Unit, Registers, L1 cache, L2 cache**
- All cores on the chip **share: L3 cache, main memory (RAM), memory bus**
- Example — Intel Core i7 with 8 cores:
 - **Core 0:** its own ALU + CU + Registers + L1 + L2
 - **Core 1:** its own ALU + CU + Registers + L1 + L2
 - ...and so on for Core 2 through Core 7
 - **All 8 cores share:** one L3 cache and one path to RAM
- **Key point:** each core can independently execute a different program — simultaneously

Multi-core Processor — The Same Core, Repeated



What Is Private vs What Is Shared

Private — Each Core Has Its Own

- Control Unit (CU)
- Arithmetic Logic Unit (ALU)
- Registers (~16-32 per core)
- L1 Cache (32-64 KB, ≈ 1 ns)
- L2 Cache (256 KB-1 MB, ≈ 5 ns)
 - Private means: no coordination
 - needed with other cores for
 - these resources

Shared — All Cores Use Together

- L3 Cache (8-64 MB, ≈ 20 ns)
- Main Memory / RAM (≈ 100 ns)
- Memory Bus (pathway to RAM)

Shared means: if two cores try to update the same data, they must coordinate — this is where **parallel programming gets tricky**

3

The Cache Hierarchy

Bridging the enormous speed gap between CPU and memory

The Speed Gap — Why Caches Are Essential

- **Recall from: the Memory Wall** — CPU is 330× faster than RAM
- Specifically:
 - A CPU core completes one arithmetic operation in ≈ 0.3 nanoseconds
 - Fetching data from main memory (RAM) takes ≈ 100 nanoseconds
 - That is a 330× difference in speed
- What happens without caches?
 - The CPU finishes a computation in 0.3 ns
 - Then waits 100 ns for the next piece of data to arrive from RAM
 - The processor is IDLE 99% of the time — a massive waste
- Caches are small, very fast memory placed between the CPU and RAM
 - They store copies of recently used data close to the processor
 - When the CPU needs data, it checks cache first — much faster than RAM

Why Caches Work — Two Principles of Locality

- Caches are effective because programs follow two predictable patterns:
- **Temporal Locality** — recently used data will likely be used again soon
 - **Example:** a loop variable i is used on every iteration
 - The CPU reads i , uses it, updates it — and reads it again next iteration
 - Keeping i in cache means it is always fast to access
- **Spatial Locality** — if you used data at address X , you will likely use $X+1$ next
 - **Example:** reading an array $A[0], A[1], A[2]...$ in order
 - When $A[0]$ is loaded, the hardware also loads $A[1]$ through $A[15]$ into cache
 - Next iteration: $A[1]$ is already in cache — no need to go to RAM
- These two patterns together make caches work very well for most programs
- Breaking them (e.g. random array access) kills cache effectiveness

Cache Hierarchy = A Library at Different Distances

You need to refer some notes about a topic. Where you store them determines how fast you can access them.

- **Registers** (< 1 ns) = Numbers written on your handbook — instant, very few
- **L1 Cache** (32-64 KB, 1 ns) = Textbooks open on your desk — one reach away
- **L2 Cache** (256KB-1MB, 5 ns) = Books on the shelf behind you — short turn
- **L3 Cache** (8-64 MB, 20 ns) = Library in your building — 2-minute walk
- **RAM** (8-128 GB, 100 ns) = City library — 30-minute drive, much slower
 - **Cache HIT:** data is on your desk — you get it instantly
 - **Cache MISS:** data is in the city library — you must drive there and wait
- Good code keeps data 'on the desk' — **sequential array** access does this
- Bad code forces 'city library trips' — **random access** does this

Cache Hierarchy — Numbers You Must Know

Level	Size	Speed	Scope	Who Controls It
Registers	< 1 KB	< 1 ns	Private to core	Compiler — automatic
L1 Cache	32–64 KB	≈ 1 ns	Private to core	Hardware — automatic
L2 Cache	256 KB–1 MB	≈ 5 ns	Private to core	Hardware — automatic
L3 Cache	8–64 MB	≈ 20 ns	Shared (all cores)	Hardware — automatic
RAM (DRAM)	8–128 GB	≈ 100 ns	Shared (all cores)	OS + hardware
SSD / Disk	256 GB+	≈ 100,000 ns	Shared (all)	OS

Cache Hit vs Cache Miss — What Actually Happens

- The cache operates in fixed-size blocks called cache lines (typically 64 bytes)
- When your program accesses $A[0]$, the cache does NOT load just $A[0]$
 - It loads a full 64-byte cache line: $A[0]$ through $A[15]$ (for int arrays)
 - This exploits spatial locality — the next 15 values are now in cache
- **Cache HIT:** CPU needs data that is already in cache
 - Result: data arrives in 1-5 ns (from L1 or L2)
 - Cost: almost nothing — the fast path

Cache Hit vs Cache Miss — What Actually Happens

Cache MISS: CPU needs data that is NOT in cache

- Result: must fetch from RAM — takes ≈ 100 ns
- The CPU STALLS — it cannot proceed until data arrives
- Sequential access ($A[0]$, $A[1]$, $A[2]$...): mostly HITS — exploits spatial locality
- Random access ($A[\text{random_index}]$): mostly MISSES — each access may go to RAM

4

The Multi-core Challenge

When two cores cache the same data — who has the real value?

Cache Coherence — The Stale Data Problem

- Each core has its own private L1 and L2 caches
- What happens when **two cores need the same variable**?
 - Both cores load their own copy into their own private caches
- A problem arises when one core modifies its copy:
 - Core 0 reads variable $X = 5 \rightarrow$ Core 0's L1 cache: $X = 5$
 - Core 1 reads variable $X = 5 \rightarrow$ Core 1's L1 cache: $X = 5$
 - Core 0 writes $X = 10 \rightarrow$ Core 0's L1 cache: $X = 10$
 - Core 1's L1 cache still shows $X = 5 \rightarrow$ STALE, **WRONG**
- If Core 1 now reads X , it gets the old value 5 — **a silent error**
- This is the **cache coherence problem** — it must be solved by the hardware

MESI Protocol — How Hardware Solves Cache Coherence



- The hardware tracks every 64-byte cache block with one of four states:
 - M — Modified: This cache has the only copy, and it has been changed. RAM is out of date.
 - E — Exclusive: This cache has the only copy, but it has not been changed. RAM is upto-date.
It can be modified easily
 - S — Shared: Multiple cores have this block. Nobody has changed it.
If some core want to modify, It must first INVALIDATE all other copies.
 - I — Invalid: current cache copy of the core is stale. So it needs to fetch fresh (new) data.
- All transitions happen automatically in hardware — not required to write MESI code
- **Cost:** every WRITE operation may send a message to other cores to invalidate

More shared data between threads = more invalidation messages = more overhead

False Sharing — The Invisible Performance Trap

False sharing occurs when two threads independently modify different variables that happen to occupy the same 64-byte cache line, causing unnecessary cache coherence traffic.

- The cache operates on 64-byte cache lines — not individual bytes
- Even if Thread 0 only needs counter[0] and Thread 1 only needs counter[1],
 - if both variables are in the same 64-byte block — they share a cache line
- Every write by Thread 0 causes the entire cache line to be marked Modified
- This INVALIDATES Thread 1's copy — even though Thread 1's variable is untouched
- Every write by Thread 1 causes the same — invalidates Thread 0's copy
- The cache line bounces back and forth between cores continuously

Result: a parallel program that should be independent can run SLOWER than sequential

- Called 'false' sharing because the variables are logically independent, but the hardware treats them as shared (same cache block)

KEY TAKEAWAYS

1. A **CPU core** = **ALU** (computes) + **CU** (directs) + **Registers** (instant storage)
2. **Multi-core** = same complete core duplicated; all cores share L3 cache and RAM
3. **Cache hierarchy**: **L1**(1ns) → **L2**(5ns) → **L3**(20ns) → **RAM**(100ns) — 100× gap
4. **Sequential array** access exploits spatial locality → **mostly cache hits** → fast
5. **Random access** breaks locality → **mostly cache misses** → 5-10× slower
6. **Cache coherence**: hardware (MESI) keeps caches consistent — at a cost
7. **False sharing**: independent data in the same 64-byte line causes slowdown

